Date:22/10/2026

C02 ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

DSA0308-NLP

Reg no:192472263 **Name. V. Gagan Kumar Reddy**

1. Morphological Analysis for Document Indexing

Aim:

To design a morphological analysis pipeline that decomposes the words connected, connecting, and connection into root and suffix components and normalizes them for document retrieval.

Methodology

1. Identify the root word as connect.
2. Separate the suffix from each input word.
3. Classify the suffix as inflectional or derivational.
4. Normalize all words to the common base form connect.

Original word	Root	Suffix	Classification	Normalized form
connected	connect	-ed	Inflectional	connect
connecting	connect	-ing	Inflectional	connect
connection	connect	-ion	Derivational	connect

Python program:

```
words = ["connected", "connecting", "connection"]


rules = {
    "connected": ("connect", "-ed", "Inflectional"),
    "connecting": ("connect", "-ing", "Inflectional"),
    "connection": ("connect", "-ion", "Derivational")
}


print(f"{'Word':<15}{'Root':<12}{'Suffix':<10}{'Type':<15}{'Normalized'}")
print("-" * 65)


for word in words:
    root, suffix, kind = rules[word]
    print(f"{word:<15}{root:<12}{suffix:<10}{kind:<15}{root}")
```

Sample output:

Word	Root	Suffix	Type	Normalized
connected	connect	-ed	Inflectional	connect
connecting	connect	-ing	Inflectional	connect
connection	connect	-ion	Derivational	connect

Result:

The program identifies the morphological components of the three words and maps them to the common normalized form connect,

improving consistency in document indexing and information retrieval.

2. Morphological Parsing for Sentiment Analysis

Aim:

To identify prefixes, suffixes, and base forms in unhappy, happiness, and happily, while classifying their morphological transformations.

Methodology:

1. Identify the base form happy.
2. Detect the prefix un- and suffixes -ness and -ly.
3. Classify each transformation as derivational or inflectional.
4. Normalize the words to the common semantic base happy.

Original word	Prefix	Base	Suffix	Classification	Normalized root
unhappy	un-	happy	None	Derivational	happy
happiness	None	happy	-ness	Derivational	happy
happily	None	happy	-ly	Derivational	happy

The spelling changes from y to i in happiness and happily. All three transformations are derivational because they create different meanings or grammatical word categories rather than merely marking tense, number, or person.

Python program:

```
words = ["unhappy", "happiness", "happily"]

rules = {
    "unhappy": ("un-", "happy", "None", "Derivational"),
    "happiness": ("None", "happy", "-ness", "Derivational"),
    "happily": ("None", "happy", "-ly", "Derivational")
}

print(f"{'Word':<15}{'Prefix':<10}{'Base':<10}{'Suffix':<10}{'Type':<15}{'Normalized'}")
print("-" * 80)

for word in words:
    prefix, base, suffix, kind = rules[word]

print(f"{word:<15}{prefix:<10}{base:<10}{suffix:<10}{kind:<15}{base}")
```

Sample output:

Word	Prefix	Base	Suffix	Type	Normalized
unhappy	un-	happy	None	Derivational	happy
happiness	None	happy	-ness	Derivational	happy
happily	None	happy	-ly	Derivational	happy

Result:

The program extracts the base word happy from all three inputs. This helps sentiment analysis systems group related words while retaining the understanding that unhappy has a negative meaning.

3. Stemming-Based Preprocessing for a Search Engine

Aim

To develop a stemming-based preprocessing module that removes suffixes from played, player, and playing and identifies their stems for efficient search and retrieval.

Methodology

1. Identify the original word.
2. Remove the appropriate suffix using predefined stemming rules.
3. Distinguish grammatical inflections from word-formation changes.
4. Normalize the words to the common indexing form play.

Python program:

```
words = ["played", "player", "playing"]

rules = {
    "played": ("play", "-ed", "Inflectional"),
```

```
    "player": ("play", "-er", "Derivational"),
    "playing": ("play", "-ing", "Inflectional")
}

print(f"{'Word':<12}{'Stem':<10}{'Removed
Affix':<15}{'Type':<15}{'Normalized'}")
print("-" * 65)

for word in words:
    stem, affix, kind = rules[word]
    print(f"{word:<12}{stem:<10}{affix:<15}{kind:<15}{stem}")
```

Sample output:

Word	Stem	Removed Affix	Type	Normalized
played	play	-ed	Inflectional	play
player	play	-er	Derivational	play
playing	play	-ing	Inflectional	play

Result:

The program extracts play from all three input words. This allows a search engine to retrieve documents containing related forms of the same word, even when their original spellings differ.

4. Finite-State Morphological Parsing for Verb Inflections

Aim:

To implement a finite-state morphological parser that processes writes, writing, and written, identifies regular and irregular verb forms, and normalizes them to write.

Methodology

A finite-state model uses states and transitions to analyze word forms.

Morphological breakdown

Word	Root	Morphological breakdown	Classification	Normalized form
writes	write	write + -s	Regular inflection	write
writing	write	writ(e) + -ing; drop final e	Regular inflection	write
written	write	Irregular past participle	Irregular inflection	write

Python program:

```
words = ["writes", "writing", "written"]


transitions = {
    "writes": ("S0 -> S1 -> S2 -> S5", "write + -s",
               "Regular inflection", "write"),
    "writing": ("S0 -> S1 -> S3 -> S5", "writ(e) + -ing",
                "Regular inflection", "write"),
```

```
    "written": ("S0 -> S1 -> S4 -> S5", "irregular form of write",
                "Irregular inflection", "write")
}


print(f"{'Word':<12}{'State
Path':<28}{'Breakdown':<25}{'Type':<22}{'Root'}")
print("-" * 100)


for word in words:
    path, breakdown, kind, root = transitions[word]


print(f"{word:<12}{path:<28}{breakdown:<25}{kind:<22}{root}")
```

Sample Output:

Word	State Path	Breakdown	Type	Root
writes	S0 -> S1 -> S2 -> S5	write + -s	Regular inflection	write
writing	S0 -> S1 -> S3 -> S5	writ(e) + -ing	Regular inflection	write
written	S0 -> S1 -> S4 -> S5	irregular form of write	Irregular inflection	write

Result:

The finite-state model recognizes the three verb forms and maps them to write. It distinguishes regular suffix-based patterns from the irregular form written, supporting consistent linguistic processing.

Note: This is a simplified finite-state model using predefined transitions for the given dataset, rather than a general-purpose morphological parser.

5. Porter Stemmer-Based Preprocessing for Document Retrieval

Aim:

To implement a Porter Stemmer-based preprocessing module that applies suffix-removal rules to relational, relation, and relate, showing intermediate forms and final stems.

Methodology:

The Porter Stemming algorithm removes selected suffixes in a sequence of steps to produce a stem.

1. Step 1: Identify the original word and its suffix.
2. Step 2: Apply the appropriate Porter stemming rule.
3. Step 3: Record the intermediate form after each rule.
4. Step 4: Obtain the final stem for indexing.

Step-by-step stemming

Original word	Applied rule	Intermediate form	Final stem
relational	ational → ate	relate	relat
relation	Remove ion when preceded by t	relat	relat
relate	Remove ate	relat	relat

These transformations follow the classic Porter algorithm's suffix rules. The final stem relat is not necessarily a complete English word; stemming aims to group related word forms, not always produce dictionary words.

Python program:

pip install nltk

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

words = ["relational", "relation", "relate"]

steps = {
    "relational": ["relational", "relate", "relat"],
    "relation": ["relation", "relat"],
    "relate": ["relate", "relat"]
}

print(f"{'Word':<15}{'Applied Rule':<30}{'Intermediate Forms':<30}{'Final Stem'}")
print("-" * 95)

for word in words:
    stem = stemmer.stem(word)
    forms = steps[word]

    if word == "relational":
        rule = "ational -> ate; remove ate"
    elif word == "relation":
        rule = "Remove ion after t"
```

```
    else:
        rule = "Remove ate"

    intermediate = " -> ".join(forms)
    print(f"{word:<15}{rule:<30}{intermediate:<30}{stem}")
```

Sample Output:

Word	Applied Rule	Intermediate Forms	Final Stem
relational	ational -> ate;	relational -> relate -> relat	relat
relation	Remove ion after t	relation -> relat	relat
relate	Remove ate	relate -> relat	relat

Result:

The program applies Porter stemming and normalizes all three words to relat. This reduces variations in indexed terms and improves matching of related words during document retrieval.